

examen 2017 / 18-2

asignatura	código	fecha	hora inicio
Análisis y diseño con patrones	05586	06/16/2018	15:30

05586 16 06 18 EX

Pegue en este espacio una etiqueta identificativa
con su código personal
examen

Este enunciado corresponde también a las asignaturas siguientes:

- 06547 - Análisis y diseño con patrones

Ficha técnica del examen

- Comprueba que el código y el nombre de la asignatura corresponden a la asignatura matriculada.
- Sólo tienes que pegar una etiqueta de estudiante en el espacio correspondiente de esta hoja.
- No se pueden adjuntar hojas adicionales, ni realizar el examen en lápiz o rotulador grueso.
- Tiempo total: **2 horas** Valor de cada pregunta: **Indicado a la pregunta**
- En caso de que los estudiantes puedan consultar algún material durante el examen, cuáles son?
CAP En caso de poder utilizar calculadora, de qué tipo? **CAP**
- Si haya preguntas tipo test: descontar las respuestas erróneas? **SÍ** Cuánto? **indicado en la pregunta**
- Indicaciones específicas para la realización de este examen:
CAP

enunciados

examen 2017 / 18-2

asignatura	código	fecha	hora inicio
Análisis y diseño con patrones	05586	06/16/2018 15:30	

Pregunta 1 (10%)

Explica, en un máximo de cinco líneas, que es un patrón de análisis y en qué se diferencia de un patrón de diseño.

solución:

Módulo 1. Apartado 2.1

Pregunta 2 (20%)

Explica brevemente los problemas que intentan resolver el patrones Dependency Injection y Estado.

solución:

Dependency Injection: Módulo 2, apartado 4.2 State: Módulo 2, apartado 6.1

Pregunta 3 (10%)

Responda si son ciertas o falsas las siguientes afirmaciones. No es necesario que justifique su respuesta. Cada una cuenta 2,5% si se acierta y descuenta 2,5% si se falla. Las respuestas en blanco no cuentan ni descuentan puntos. La nota mínima de esta pregunta será 0.

- a) El patrón asociación histórica es un patrón de diseño
(Falso. Módulo 2, apartado 3.1)
- b) Si existe un patrón para un problema que tenemos, no necesitamos plantear otras soluciones ya que sabemos que esta es la mejor solución posible para el problema
(Falso. Módulo 1, apartado 1.6)
- c) Una clase con acoplamiento alto es más fácil de entender de manera aislada
(Falso. Modulo2, apartado 2.1)
- d) La ley de Deméter nos ayuda a mantener acoplamiento abajo
(Cert. Módulo 2, apartado 3.1)

examen 2017 / 18-2

assignatura	código	fecha	hora inicio
Análisis y diseño con patrones	05586	06/16/2018	15:30

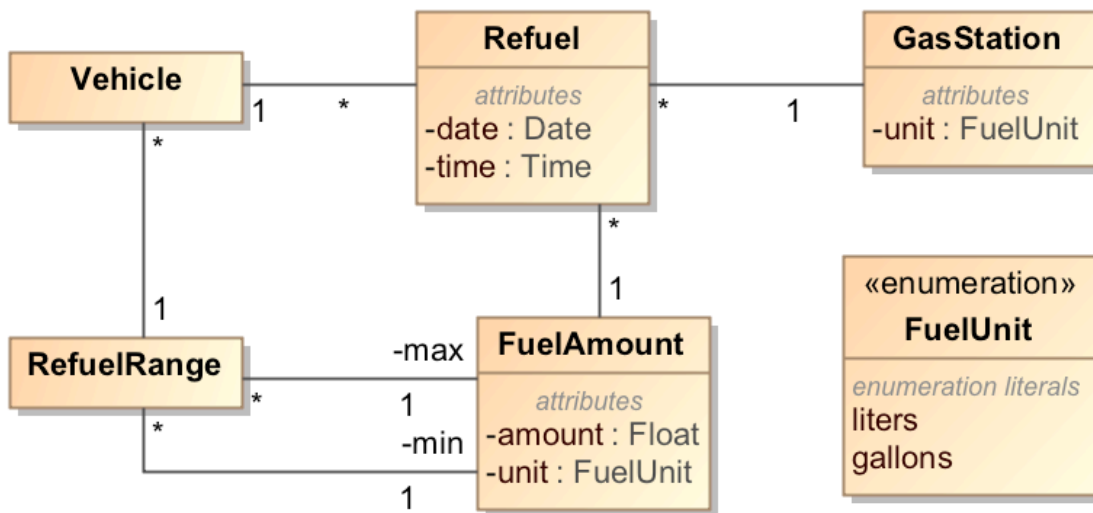
Ejercicio 1 (30%)

Supongamos que queremos desarrollar un software que registre los consumos de combustibles de una flota de vehículos industriales. Para hacerlo, cada vehículo tendrá registrados los repostajes de combustible que hace (volumen de combustible, estación de repostaje, fecha y hora).

También queremos controlar que un vehículo no pueda tener asociado un repostaje mayor que la capacidad del vehículo (para evitar el fraude) y que nunca se haga un repostaje de menos del 30% de la capacidad máxima del vehículo (para evitar tener el vehículo parado innecesariamente). Así, si un vehículo tiene un depósito de 100 litros de capacidad, nunca debería tener un repostaje de menos de 30 litros ni de más de 100 litros de combustible. Por motivos que ahora no vienen al caso pero que nos complicarán un poco la vida, algunas estaciones de repostaje reportan el volumen en litros y otros en galones, así que tendremos que hacer las conversiones necesarias. Se pide el **diagrama estático de análisis** para registrar esta información. En caso de que hayas aplicado algún patrón de análisis, indica cuál o cuáles patrones has aplicado. Haz los supuestos que creas necesarios y justifica tus decisiones.

solución:

Hemos aplicado el patrón Cantidad por la cantidad de combustible (así evitamos problemas con las diferentes unidades) y el patrón Range al rango de cantidades de combustible que debe tener asociado un vehículo:

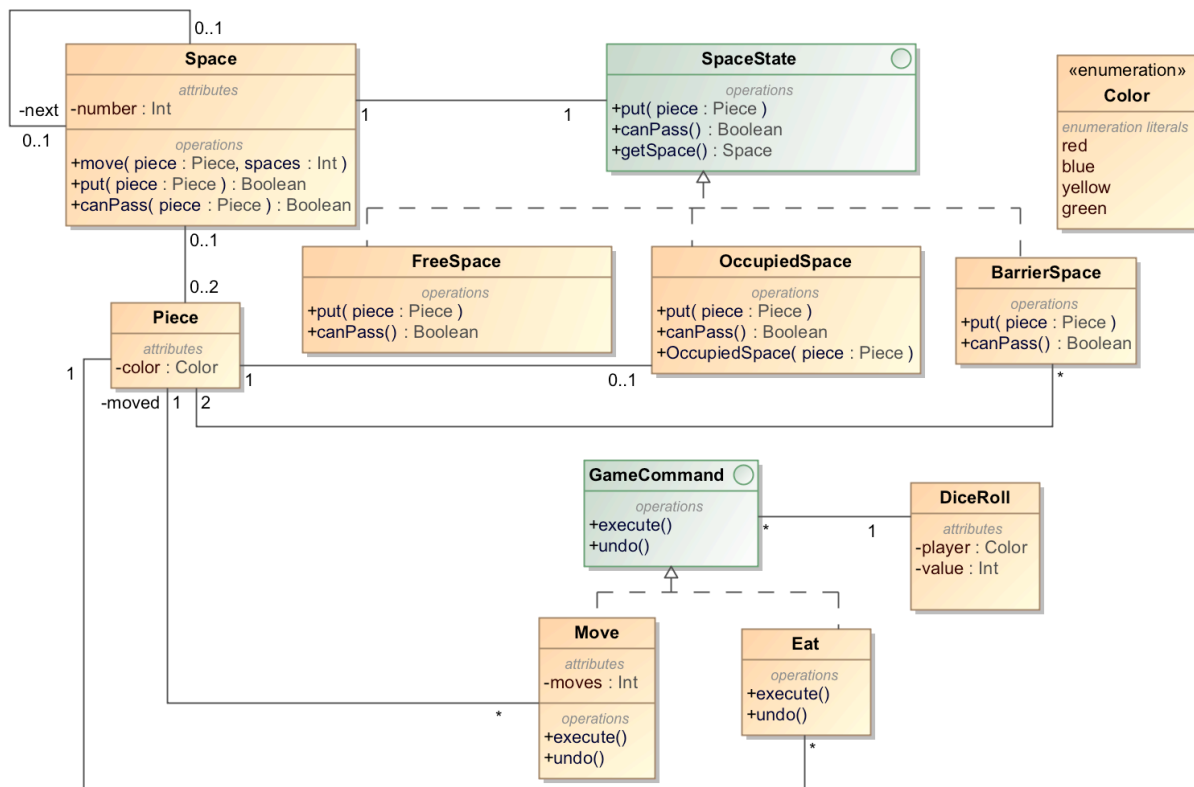


examen 2017 / 18-2

asignatura	código	fecha	hora inicio
Análisis y diseño con patrones	05586	06/16/2018 15:30	

Ejercicio 2 (30%)

Estamos desarrollando un software que implementa una versión simplificada del juego del parchís. Una casilla (Space) tiene un número y puede estar libre u ocupada. Cada ficha (Piece) puede ocupar una casilla o ninguna. Cuando se produce un tirón (DiceRoll) se obtiene un valor entre 0 y 6 y, se pueden hacer tantos movimientos (*Move*) como quiera el jugador, siempre y cuando la suma de casillas movidas que se hace sea igual al valor del dado. Si en alguno de estos movimientos nos encontramos con una barrera, tendremos que deshacer el movimiento y volver a la situación original ya que no podemos pasar. Si el movimiento finaliza en una casilla ocupada, se genera una nueva orden de tipo *Eat* que también permite mover la ficha, etc. Disponemos del siguiente diagrama de diseño:



a) ¿Qué patrones se han aplicado en este diagrama de diseño? Indique qué clases del diagrama implementan los patrones detectados.

b) Proponer el diagrama de secuencia o escriba el pseudocódigo que muestre cómo se implementa la operación

put de la clase *Space* cuando una casilla está libre (los otros estados los ignoraremos la solución para no hacer el ejercicio demasiado largo).

Cuando ponemos una pieza a una casilla que está libre, ésta pasa a ser ocupada por la pieza que hay

hemos movido y la operación *put* devuelve *true* para indicar que se ha podido completar el movimiento.

solución

a) Se han aplicado los patrones Estado (State) y Orden (Command).

las clases *Space*, *SpaceState* y sus subclases implementan el patrón Estado (State). las clases *GameCommand*, *Move* y *Eat* implementan el patrón Orden (Command).

b)

examen 2017 / 18-2

assignatura	código	fecha	<u>hora inicio</u>
Análisis y diseño con patrones	05586	06/16/2018	15:30

```
class Space {  
    private Space next; private SpaceState space;  
    public Boolean put (Piece piece) {space.put  
    (piece); return true; }  
  
    private void setState (state: SpaceState) {this.state =  
    state; }}  
  
class FREESPACE extends SpaceState {override  
    public Boolean canPass () {return true; }  
  
    override public Boolean put (Piece piece) {  
        OccupiedSpace nextState = new OccupiedSpace (piece); getSpace (). setState  
(nextState); return true; }
```